> arm's geometry

- horizontal x-y arm, with z pointing upward
- both joints rotate around the z-axis → gravity: negative z

- The XML model defines relatively permanent structure: bodies, link shapes, joint axes, cameras, actuators, and simulation settings.
- `MjData` holds changing simulation state: time, joint positions (`qpos`), joint velocities (`qvel`), and control inputs (`ctrl`).

A cylinder with half-height 0.05 extends 0.05 m above and below its center:

e.g.

```
center z = 0.05
top      = 0.05 + 0.05 = 0.10
bottom   = 0.05 - 0.05 = 0.00
                                ← ground
```

If its center were at $z = 0$, half the cylinder would be below the ground plane.

→ timestep : 0.002 → 500 steps for 1.0 s.

> arm

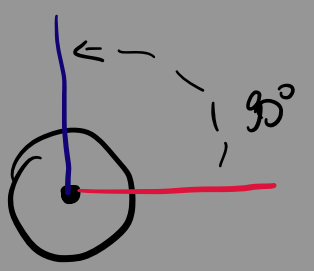
shoulder location: (0, 0, 0.13)

→ extends along +x axis

begins at shoulder ← $L_1 = 0.25$m

the range is approx. $-170° \leftrightarrow +170°$
expressed in rad.

suppose $\theta_1 = \frac{\pi}{2}$



goes from 0,25 , 0 , 0,13

to

0 , 0,25 , 0,13

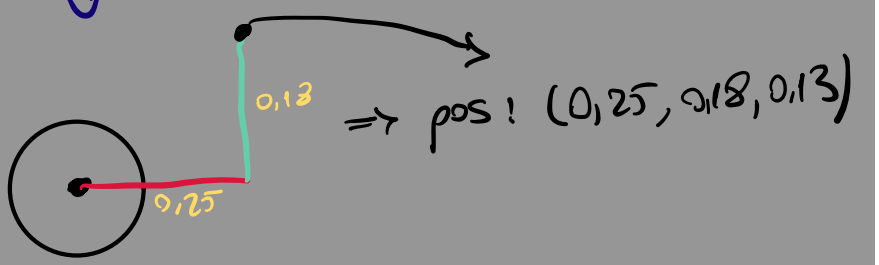
$L_2 = 0{,}18m$ will be nested inside L_1

↳ body pos.: 0,25 , 0 , 0

suppose $\theta_1 = 0$, $\theta_2 = \pi/2$

↓ points +X ⇒ so the elbow is 0,25 , 0 , 0,13

⇓



⇒ pos: (0,25, 0,18, 0,13)

↳ Kinematic hierarchy

world
- base
  - L_1 + shoulder
    - L_2 + elbow
      - end_effector + site

→ fixed target + overhead camera

For the target:

- It has no joint, so it is fixed.
- Box sizes are half-sizes.
- Its half-height and center height are both 0.03 m, so its bottom rests at $z = 0$.

For the camera:

- It is 1.2 m above the origin.
- MuJoCo cameras look along their local negative z-axis.
- `xyaxes` aligns the camera's local x and y axes with world $+x$ and $+y$, leaving it looking downward along world $-z$.
- `fovy="50"` is its vertical field of view in degrees.

applying actuator: applying input causing motion thru physics

L'motor' will apply torque to hinge.

shoulder_motor = shoulder joint

elbow_motor = elbow joint

! if ctrl=0 but qvel is positive

↳ ppos still changes even tho there is no torque / force / acceleration current vel. makes it keep moving.

- Fresh reset + zero controls + zero velocity → stationary.
- Moving arm + controls returned to zero → continues through inertia.

= MjModel: compiled, mostly fixed model description

MjData: changing simulation state for one instance of that model

Forward Kinematics

first link

$L_1 = 0.25$

shoulder angle $= \theta_1$

shoulder origin $= (0,0)$

when $\theta_1 = 0 \rightarrow (L_1, 0)$

$\theta_1 = \pi/2 \rightarrow (0, L_1)$

$\Rightarrow x_e = L_1 \cos\theta_1$

$y_e = L_1 \sin\theta_1$

second link

$L_2 = 0.18$

elbow $= (L_1\cos\theta_1, L_1\sin\theta_1)$

orientation $= \theta_1 + \theta_2$

$\Rightarrow \Delta x_2 = L_2 \cos(\theta_1 + \theta_2)$

$\Delta y_2 = L_2 \sin(\theta_1 + \theta_2)$

$$x = L_1\cos\theta_1 + L_2\cos(\theta_1+\theta_2)$$
$$y = L_1\sin\theta_1 + L_2\sin(\theta_1+\theta_2)$$

Inverse Kinematics

Solving for the elbow term:

$$\cos(\theta_2) = \frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}$$

This convention gives:

- $\theta_2 = 0$: links fully extended.
- $\theta_2 = \pi$: link 2 completely folded backward.

on ik_pd_demo

Notice something physically important: the shoulder target and initial shoulder torque were both zero, but by `0.4 s` its angle became `-0.1183 rad`.

↳ elbow motor accelerates link 2 > dynamic coupling
both links are physically connected
its reaction force also moves shoulder

once the shoulder moves away from zero, its own PD controller reacts

Current position (M:1 0-3) :

1. Milestone 0 — MuJoCo foundations
   You built a two-link arm in the horizontal x-y plane. Both joints rotate around z, joint angles are relative, and motors command torque—not angle. You explored `qpos`, `qvel`, `ctrl`, simulation stepping, inertia, and dynamic coupling.
2. Milestone 1 — Forward kinematics
   Joint angles became Cartesian end-effector coordinates:
   $$x = L_1 \cos\theta_1 + L_2 \cos(\theta_1 + \theta_2)$$
   $$y = L_1 \sin\theta_1 + L_2 \sin(\theta_1 + \theta_2)$$
   Manual results were verified against MuJoCo.
3. Milestone 2 — Inverse kinematics
   Cartesian targets became joint-angle solutions. You implemented reachability, both elbow configurations, singular-boundary handling, and verification through both FK and MuJoCo. Current reachability is geometric and does not yet enforce MuJoCo's joint limits.
4. Milestone 3 — Feedback control
   You progressed from P control to PD control:
   $$\tau = K_p(q_d - q) - K_d\dot{q}$$
   P reacts to position error; D brakes motion. You added torque saturation, controlled both dynamically coupled joints, fed IK results into the controller, and reached the Cartesian target with about `0.685 mm` error.

The current complete pipeline is:

$$\text{Cartesian target} \rightarrow \text{IK} \rightarrow \text{desired joint angles} \rightarrow \text{PD torque control} \rightarrow \text{MuJoCo arm}$$

pixel → world mapping

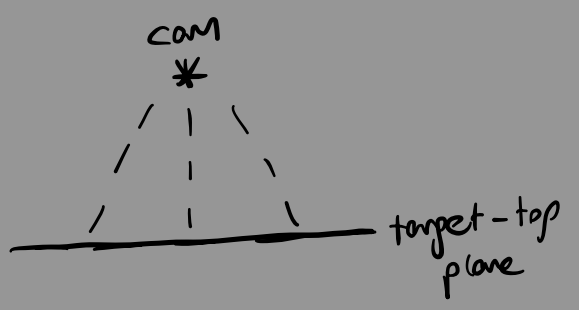
> an axis aligned overhead pinhole camera viewing a horizontal plane

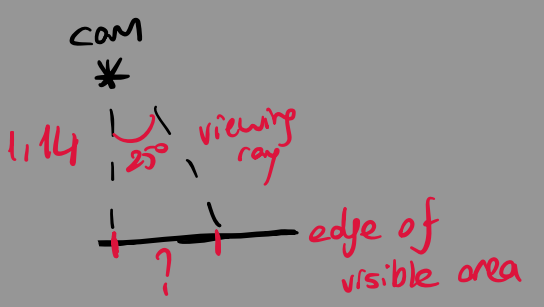
```
camera_z = 1.20 m
target_top_z = 0.06 m
camera-to-plane distance = 1.14 m
vertical field of view = 50°
image height = 480 px
```

> the camera's visible vertical world span at that plane comes from the pinhole triangle:

$$H_{world} = 2d\tan\left(\frac{fovy}{2}\right)$$

↳ $H_{world} = 2(1{,}14)\cdot\tan 25°$





⇒ purpose: translate what cam. sees into coords. the robot understands

$H_{world} = 2 \cdot 1{,}14 \tan\left(\frac{50°}{2}\right)$

$\approx 1{,}063\ m$

$\Rightarrow \frac{1{,}063}{480} \approx \boxed{2{,}215\ mm}$

> $H_{world} = 2d\tan\left(\frac{fovy}{2}\right)$

$scale = \frac{H_{world}}{image\_height}$

$x = x_{cam} + \left(u - \frac{w-1}{2}\right)s$

$y = y_{cam} - \left(v - \frac{H-1}{2}\right)s$

1. Pinhole/FOV formula → meters per pixel.
2. Subtract image center → pixel displacement.
3. Multiply displacement by scale → displacement in meters.
4. Add camera x, y → absolute world position.
5. Reverse the v sign because image-down and world-$+y$ point oppositely.

offset
u - v

↳ closed-loop perception error

> target never moves but centroid does

one camera observation: 0.586 mm error

repeated camera observation: 8.856 mm error

↳ TL;DR
small oscillation happens because overhead cam. doesn't see the whole cube and can't estimate the centroid.

⚠ > the green end-effector marker and arm cover parts of the red cube

> since the detector selects largest red contour, a partially visible fragment becomes the "target" shifting its centroid

→ one-cam. observation

1. Arm starts away from target.
2. Camera sees the complete red cube.
3. Detector finds the center accurately.
4. That target estimate is frozen.
5. Arm moves toward the frozen position.

→ repeated camera observation

1. Camera sees complete cube → correct centroid.
2. Arm moves closer.
3. Arm passes between overhead camera and cube.
4. Green marker/arm hides some red pixels.
5. Detector calculates the center of only the visible red portion.
6. Target estimate shifts.
7. Arm follows the shifted estimate.

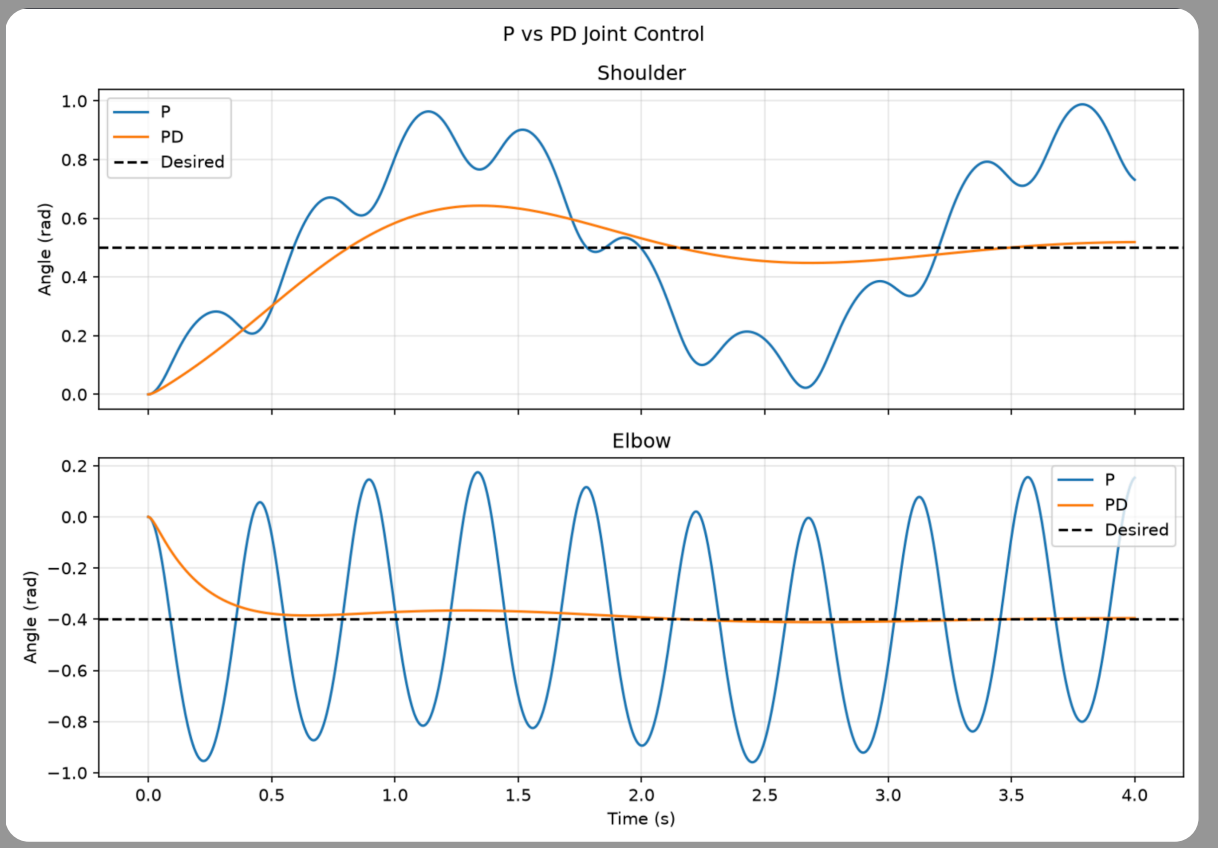
→ after Mil 7 completed

target moves and camera detects its new location and produces different IK angles

the robot responds using vision rather than simulator coords.

=> Experiments

> P vs PD



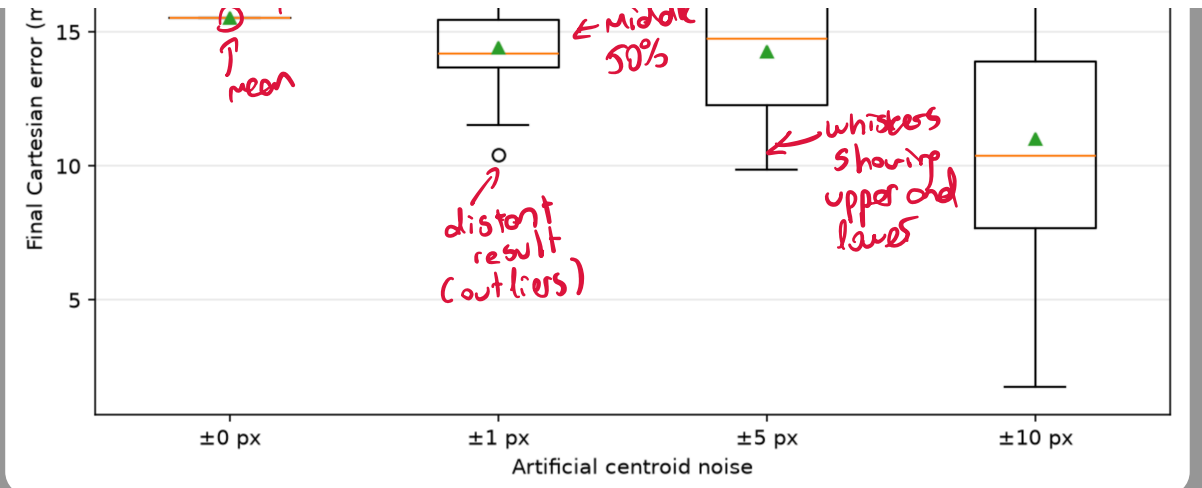
> Perception Noise

Why can noise occasionally improve accuracy? The clean detector already has a systematic centroid bias from arm occlusion. Random noise sometimes shifts that biased centroid closer to the cube's real center by chance. Other times it shifts it farther away. That is why ±10 px produced both the best and worst individual noisy results.

With 20 trials:

Noise	Mean error	Standard deviation
0 px	15.513 mm	0.000 mm
±1 px	14.405 mm	1.903 mm
±5 px	14.284 mm	2.669 mm
±10 px	11.015 mm	5.508 mm

> Pixel noise increased outcome variability and reduced repeatability, even though it sometimes counteracted the existing occlusion bias and lowered mean final error.



> Control frequency

how often the controller reads joint state and calculates new torques.

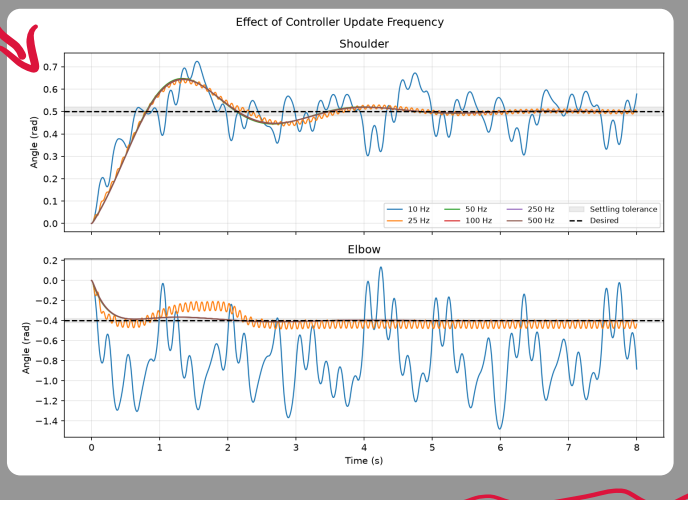
Control rate (Hz)	Joint RMSE (rad)	Final error (rad)	Shoulder overshoot (rad)	Elbow overshoot (rad)	Settling time (s)
10	0.357133	0.489350	0.224337	1.081480	Not settled within 8 s
25	0.090210	0.034140	0.147676	0.084315	Not settled within 8 s
50	0.081228	0.001125	0.147789	0.012036	4.114
100	0.081562	0.001006	0.145095	0.011602	3.234
250	0.081770	0.000936	0.143518	0.011352	3.240
500	0.081841	0.000912	0.142999	0.011271	3.242

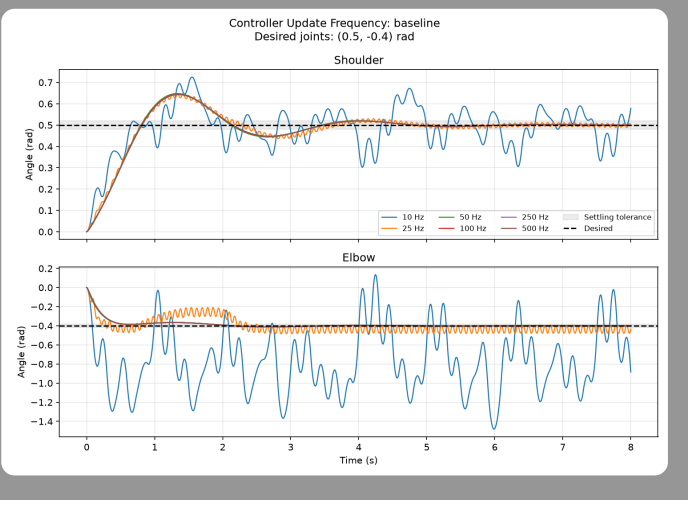
smaller movement did not auto. perform better.

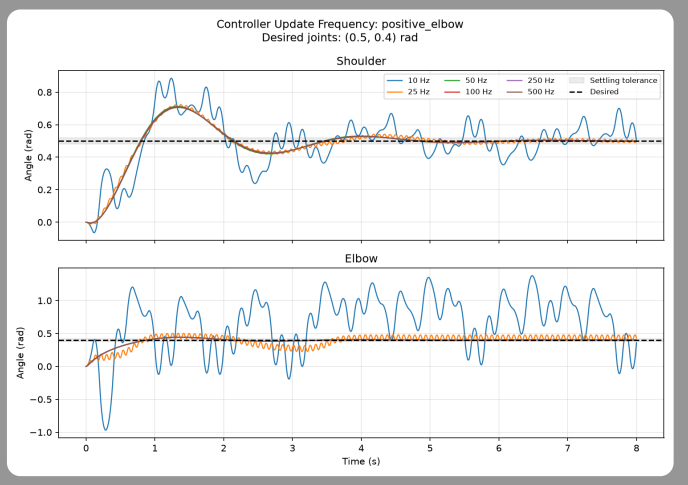
settled

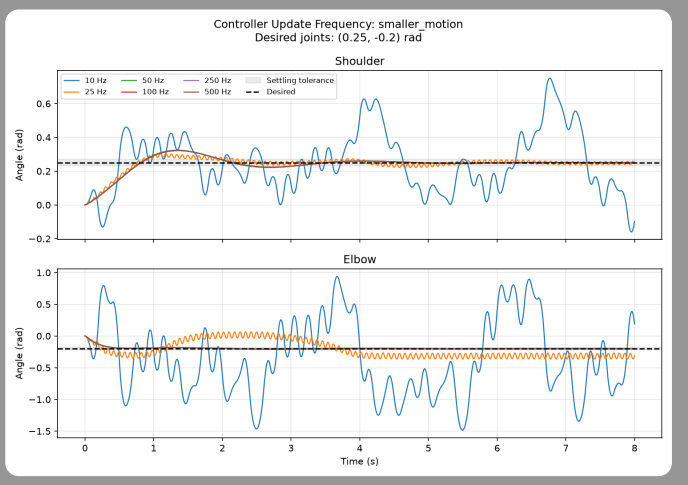
little improvement

Additional configs









> Perception Latency

the delay between capturing an observation and making it available to the controller.

Added latency (ms)	Tracking RMSE (mm)	Peak error (mm)	Final error (mm)	Missed detections
0	15.601	22.471	14.673	0
10	15.400	21.427	16.745	0
20	15.261	22.744	17.096	0
50	14.473	23.677	15.628	0
100	14.655	25.287	15.927	0
200	14.401	31.877	3.803	0

* 200 ms increased peak error 41.9%
but not rmse / final

Latency did not worsen every metric